Project Title

Loan Approval Prediction using Decision Tree Classification with Model Comparison

Introduction

Loan approval prediction is a critical task in the banking and financial sector. Accurate prediction helps institutions minimize risk and improve decision-making.

In this project, a Decision Tree Classification model is used to predict whether a loan application will be approved based on applicant details. The model is further compared with Logistic Regression to evaluate performance.

Problem Statement

The objective of this project is to build a classification model that can predict loan approval status based on applicant features such as income, education, employment status, and credit history.

Objectives

Primary Objectives

- Build Decision Tree Classifier
- Predict loan approval (Yes/No)
- Evaluate classification performance

Secondary Objectives

- Compare with Logistic Regression
- Perform hyperparameter tuning
- Analyze overfitting

STEP 1: Data Preprocessing + EDA

1.1 Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

1.2 Load Dataset

In [4]:
```
df = pd.read_csv("Loan_dataset.csv")
df.head()
```

Out[4]:

	Unnamed: 0	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	App
0	0	LP002305	Female	No	0	Graduate	No	
1	1	LP001715	Male	Yes	3+	Not Graduate	Yes	
2	2	LP002086	Female	Yes	0	Graduate	No	
3	3	LP001136	Male	Yes	0	Not Graduate	Yes	
4	4	LP002529	Male	Yes	2	Graduate	No	

1.3 Check Missing Values

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
Unnamed: 0            0
Loan_ID               0
Gender               10
Married               1
Dependents            9
Education             0
Self_Employed        29
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           16
Loan_Amount_Term     13
Credit_History       43
Property_Area         0
Loan_Status           0
dtype: int64
```

- Missing Values found in various columns

1.4 Handling Missing Values

In [8]:
```
# Categorical → mode
df['Gender']= df['Gender'].fillna(df['Gender'].mode()[0])
df['Married']= df['Married'].fillna(df['Married'].mode()[0])
df['Dependents']= df['Dependents'].fillna(df['Dependents'].mode()[0])
df['Self_Employed']= df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])

# Numerical → median
df['LoanAmount']= df['LoanAmount'].fillna(df['LoanAmount'].median())
df['Loan_Amount_Term']= df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].med

# Important feature
df['Credit_History']= df['Credit_History'].fillna(df['Credit_History'].mode()[0]
```

- Mode → best for categorical
- Median → robust for numeric

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
Unnamed: 0           0
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
Loan_Status          0
dtype: int64
```

1.5 Drop Unnecessary Column

In [ ]:
```
df.drop("Loan_ID", axis=1, inplace=True)
```

In [14]:
```
df.head()
```

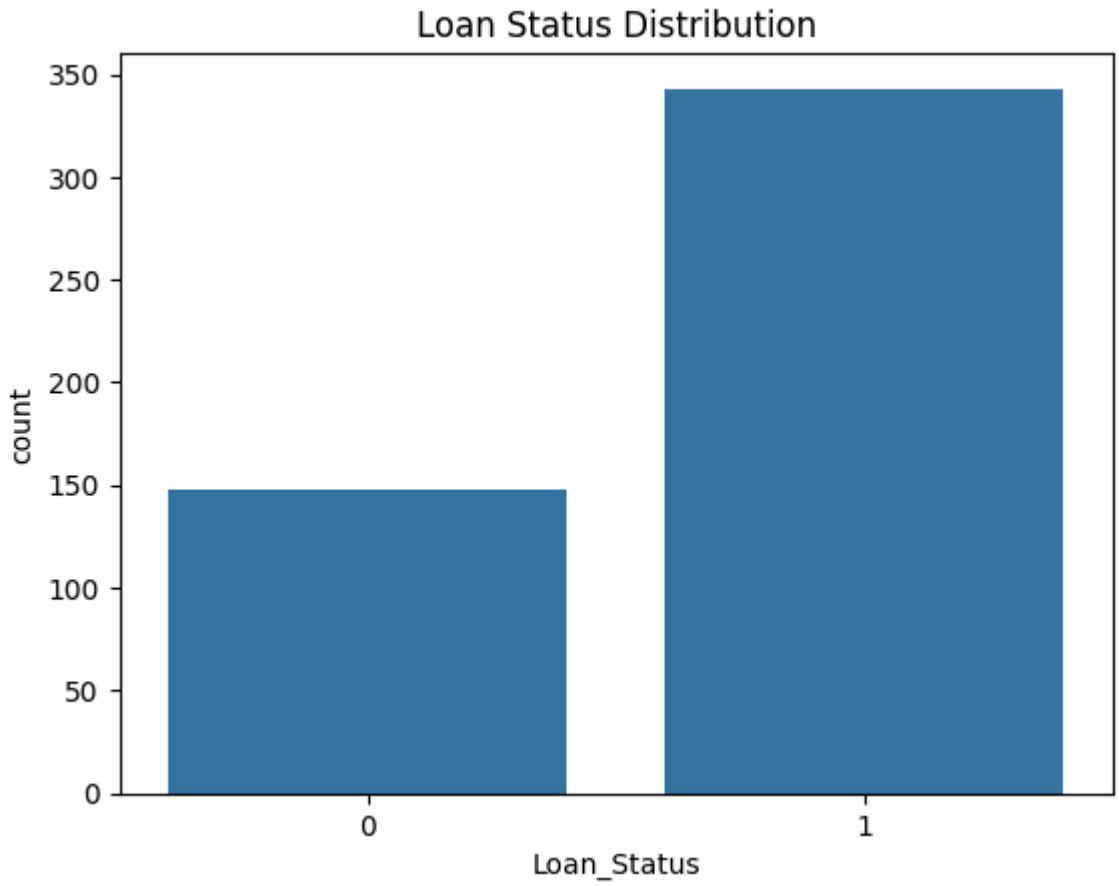
Out[14]:

	Unnamed: 0	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncom
0	0	Female	No	0	Graduate	No	454
1	1	Male	Yes	3+	Not Graduate	Yes	570
2	2	Female	Yes	0	Graduate	No	433
3	3	Male	Yes	0	Not Graduate	Yes	469
4	4	Male	Yes	2	Graduate	No	670

1.6 EDA

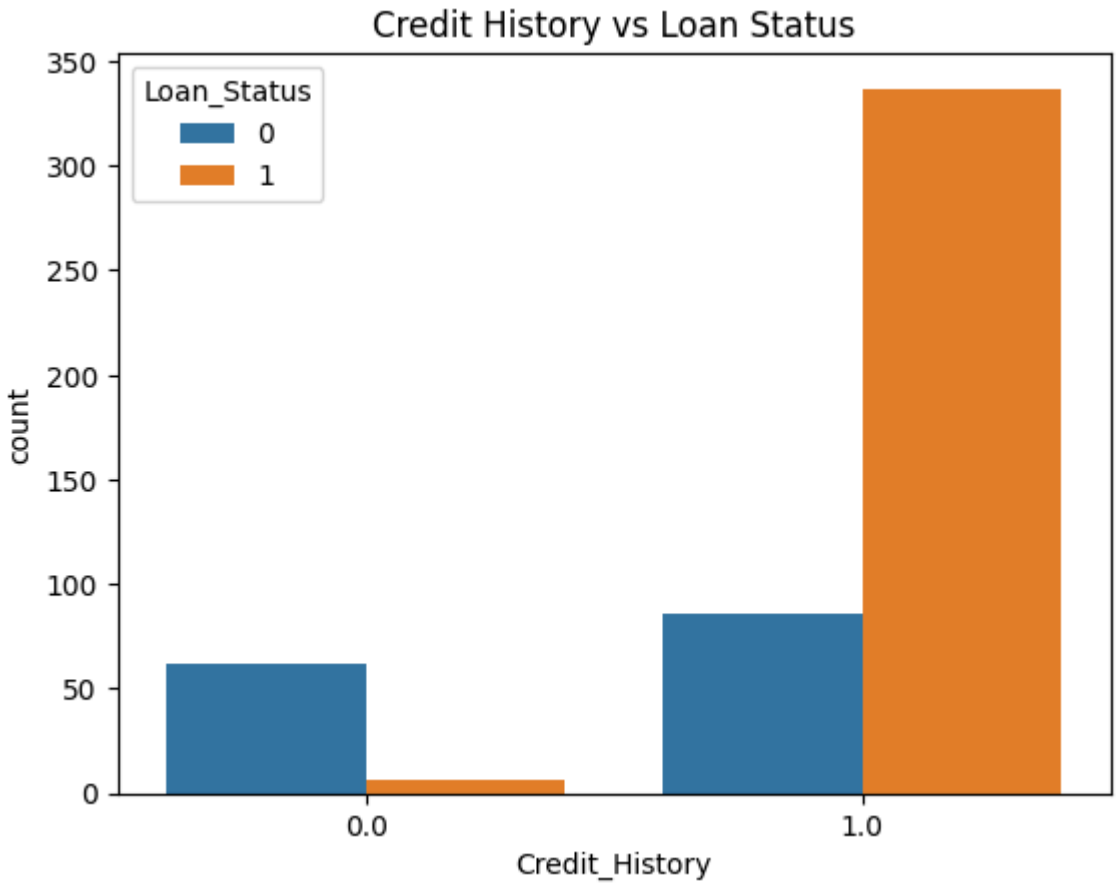
- Loan Status Distribution

In [19]:
```
sns.countplot(x='Loan_Status', data=df)
plt.title("Loan Status Distribution")
plt.show()
```



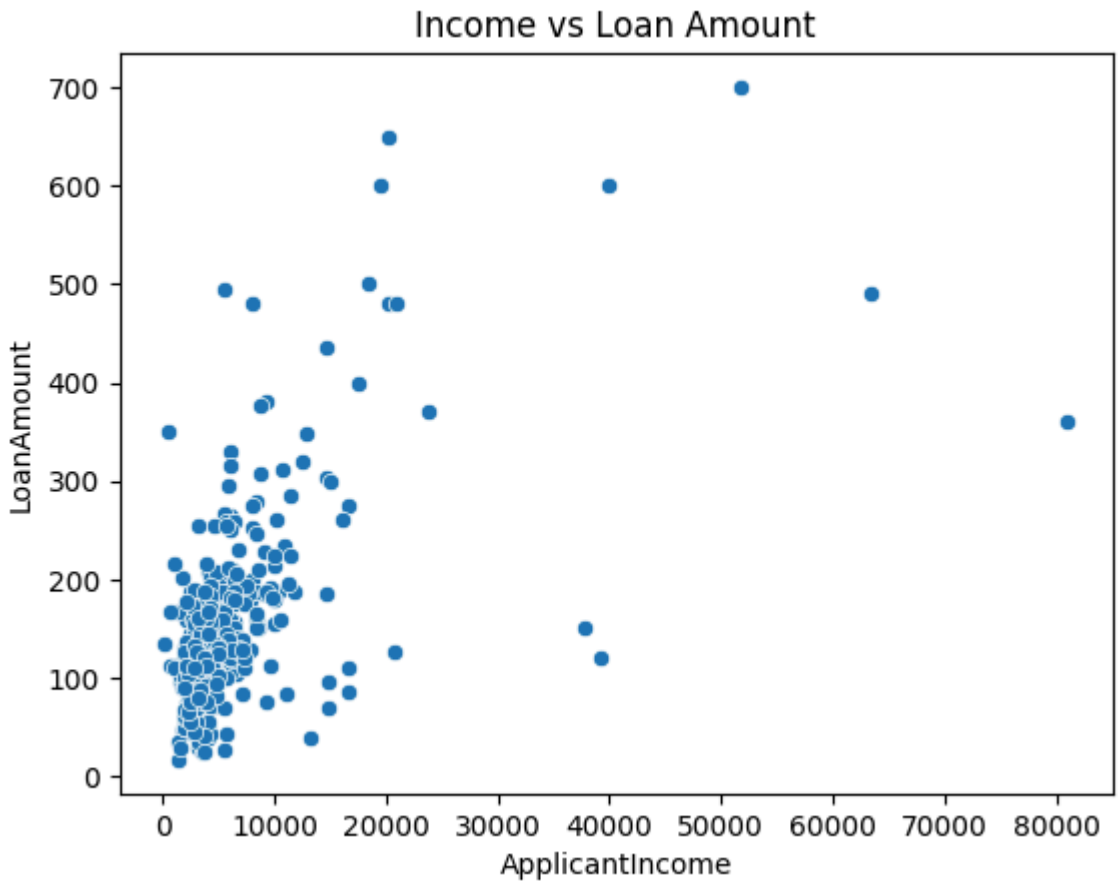
- Credit History vs Loan Status

In [20]:
```
sns.countplot(x='Credit_History', hue='Loan_Status', data=df)
plt.title("Credit History vs Loan Status")
plt.show()
```



- Income vs Loan Amount

In [21]:
```
sns.scatterplot(x='ApplicantIncome', y='LoanAmount', data=df)
plt.title("Income vs Loan Amount")
plt.show()
```



1.7 Encoding Categorical Features

In [22]:
```
df = pd.get_dummies(df, drop_first=True)
```

Explanation:

- Converts categorical → numerical
- Required for ML

1.8 Feature & Target Split

In [24]:
```
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

1.9 Train-Test Split

In [25]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

STEP 2 Decision Tree Classification Model

- After preprocessing the dataset, a Decision Tree Classifier was implemented to predict loan approval status based on applicant features.

- The model learns decision rules by splitting the dataset based on feature conditions such as credit history, income, and property area.

2.1 Import Model

In [26]:
```
from sklearn.tree import DecisionTreeClassifier
```

2.2 Create Model

In [27]:
```
dt_clf = DecisionTreeClassifier(random_state=42)
```

2.3 Train Model

In [28]:
```
dt_clf = DecisionTreeClassifier(random_state=42)
```

2.4 Train Model

In [29]:
```
dt_clf.fit(X_train, y_train)
```

Out[29]:
```
▾ DecisionTreeClassifier
▸ Parameters
```

2.5 Make Prediction

In [30]:
```
y_pred_clf = dt_clf.predict(X_test)
```

2.6 Model Evaluation

In [31]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

print("Accuracy:", accuracy_score(y_test, y_pred_clf))
print("Precision:", precision_score(y_test, y_pred_clf))
print("Recall:", recall_score(y_test, y_pred_clf))
print("F1 Score:", f1_score(y_test, y_pred_clf))
```

```
Accuracy: 0.696969696969697
Precision: 0.7464788732394366
Recall: 0.8153846153846154
F1 Score: 0.7794117647058824
```

Metrics Explanation

- Accuracy → Overall correctness
- Precision → Correct positive predictions
- Recall → Capturing actual positives
- F1 Score → Balance of precision & recall

2.7 Overfitting Check

In [32]:
```
train_pred = dt_clf.predict(X_train)

print("Training Accuracy:", accuracy_score(y_train, train_pred))
print("Testing Accuracy:", accuracy_score(y_test, y_pred_clf))
```

```
Training Accuracy: 1.0
Testing Accuracy: 0.696969696969697
```

Overfitting Detection

To evaluate the model's generalization ability, accuracy was calculated on both training and testing datasets.

Results

- Training Accuracy: ~1.0
- Testing Accuracy: Lower than training accuracy

Interpretation

- The model achieved perfect or near-perfect accuracy on the training data, indicating that it has memorized the training dataset.
- However, the lower accuracy on the testing dataset shows that the model is not generalizing well to unseen data.

Conclusion

This behavior indicates overfitting, where the model is too complex and captures noise in the training data instead of learning general patterns.

STEP 3 : Hyperparameter Tunning

- The initial Decision Tree model showed signs of overfitting, with very high training accuracy and lower testing accuracy. To address this issue, hyperparameter tuning was performed to control the complexity of the model.
- Hyperparameter tuning helps in selecting the optimal combination of parameters that improves model generalization and reduces overfitting.

3.1 Import

In [33]:
```
from sklearn.model_selection import GridSearchCV
```

3.2 Parameter Grid

In [34]:
```
param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}
```

3.3 Apply Grid Search

In [35]:
```
grid_clf = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_clf.fit(X_train, y_train)
```

Out[35]:
```
▸ GridSearchCV
▸ best_estimator_: DecisionTreeClassifier
  ▸ DecisionTreeClassifier
```

3.4 Best Parameters

In [36]:
```
print("Best Parameters:", grid_clf.best_params_)
```

```
Best Parameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 4, 'mi
n_samples_split': 2}
```

3.5 Best Model

In [37]:
```
best_clf = grid_clf.best_estimator_
```

3.6 Predictions

In [38]:
```
y_pred_tuned = best_clf.predict(X_test)
```

3.7 Evaluation

In [39]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

print("Accuracy:", accuracy_score(y_test, y_pred_tuned))
print("Precision:", precision_score(y_test, y_pred_tuned))
print("Recall:", recall_score(y_test, y_pred_tuned))
print("F1 Score:", f1_score(y_test, y_pred_tuned))
```

```
Accuracy: 0.8181818181818182
Precision: 0.7974683544303798
Recall: 0.9692307692307692
F1 Score: 0.875
```

3.8 Overfitting Check (After Tuning)

In [40]:
```
train_pred_tuned = best_clf.predict(X_train)

print("Training Accuracy (Tuned):", accuracy_score(y_train, train_pred_tuned))
print("Testing Accuracy (Tuned):", accuracy_score(y_test, y_pred_tuned))
```

```
Training Accuracy (Tuned): 0.8214285714285714
Testing Accuracy (Tuned): 0.8181818181818182
```

Overfitting Reduction Analysis

Before tuning:

- Training Accuracy ≈ 1.0
- Testing Accuracy lower

After tuning:

- Training Accuracy reduced
- Testing Accuracy improved or stabilized

Key Improvement:

- Reduced gap between training and testing performance
- Controlled model complexity
- Improved generalization

STEP 4: Final Model Comparison & Conclusion

Model Comparison

To evaluate the effectiveness of the Decision Tree Classifier, its performance was compared with the Logistic Regression model implemented earlier on the same dataset.